۷-۳ همگامی پوزیکس

روش‌های قفل‌گذاری بحث شده در بخش قبل برای همگامی در میان هسته مناسب بوده و لذا تنها در دسترس توسعه‌دهندگان هسته هستند. در مقایسه، API پوزیکس برای برنامه‌نویسان در سطح کاربر فراهم است و جزء هسته‌ی سیستم‌عامل خاصی نیست. (البته، بایستی نهایتاً با استفاده از ابزاری که توسط سیستم‌عامل میزان فراهم می‌شود پیاده‌سازی شوند.)

ما در این بخش، قفل‌های انحصار متقابل، سمافورها، و متغیرهای شرط را پوشش می‌دهیم که در API نخ‌های پوزیکس و پوزیکس در دسترس هستند. این APIها به‌صورت گسترده توسط توسعه‌دهندگان در سیستم‌های یونیکس، لینوکس، و macOS برای ایجاد نخ و همگامی استفاده می‌شوند.

۷-۳-۱ قفل‌های انحصار متقابل پوزیکس

قفل‌های انحصار متقابل، تکنیک اساسی همگامی مورد استفاده در نخ‌های پوزیکس را فراهم می‌کنند. قفل انحصار متقابل برای محافظت بخش‌های بحرانی کد—یعنی، هر نخ پیش از ورود به بخش بحرانی، قفل را در اختیار می‌گیرد و با خروج از بخش بحرانی، آن را رها می‌سازد—استفاده می‌شود. نخ‌های پوزیکس از نوع داده‌ی pthread_mutex_t برای قفل‌های انحصار متقابل استفاده می‌کنند. انحصار متقابل با تابع pthread_mutex_init() ایجاد می‌شود. پارامتر اول، اشاره‌گری به انحصار متقابل است. با ارسال NULL به عنوان پارامتر دوم، انحصار متقابل را با صفات پیش‌فرض آن مقداردهی می‌کنیم. این روال در زیر نشان داده می‌شود:

```
#include <pthread.h>
pthread_mutex_t mutex;

/* create the mutex lock */
pthread_mutex_init(&mutex,NULL);
```

انحصار متقابل با استفاده از توابع pthread_mutex_lock() و pthread_mutex_unlock() کسب و رها می‌گردد. اگر قفل انحصار متقابل در زمان احضار تابع pthread_mutex_lock() در دسترس نباشد، نخ فراخوانده مسدود می‌شود تا این که مالک، تابع pthread_mutex_unlock() را احضار کند. کد زیرین، حفاظت از بخش بحرانی را با قفل‌های انحصار متقابل نشان می‌دهد:

```
/* acquire the mutex lock */
pthread_mutex_lock(&mutex);

/* critical section */

/* release the mutex lock */
pthread_mutex_unlock(&mutex);
```

تمامی توابع انحصار متقابل، با عملیات درست، مقدار ۰ بر می‌گردانند؛ این توابع در صورت

وقوع خطا یک کد خطای غیر صفر بر می‌گردانند.

۷-۳-۲ سمافورهای پوزیکس

بسیاری از سیستم‌هایی که نخ‌های پوزیکس را پیاده‌سازی می‌کنند، سمافورها را نیز پشتیبانی می‌کنند، هرچند که سمافورها به عنوان بخشی از استاندارد نخ‌های پوزیکس به حساب نمی‌آیند ولی در عوض، به گسترش POSIX SEM تعلق دارند. پوزیکس دو نوع سمافور—**با نام** و **بی‌نام**— توصیف می‌کند. این دو در اصل کاملاً شبیه هم هستند، اما در چگونگی ایجاد و اشتراک میان پردازه‌ها فرق دارند. چون هر دو تکنیک رایج هستند، در اینجا هر دو را بحث می‌کنیم. لینوکس با شروع از نسخه‌ی 2.6 هسته‌اش پشتیبانی هر دوی سمافورهای با نام و بی‌نام را فراهم می‌کند.

۷-۳-۲-۱ سمافورهای با نام پوزیکس

تابع sem_open() برای ایجاد و بازکردن یک سمافور با نام پوزیکس استفاده می‌شود:

```
#include <semaphore.h>
sem_t *sem;

/* Create the semaphore and initialize it to 1 */
sem = sem_open("SEM", O_CREAT, 0666, 1);
```

ما در این نمونه، سمافور را SEM نام‌گذاری می‌کنیم. پرچم O_CREAT نشان می‌دهد که در صورتی که سمافور قبلاً وجود نداشته باشد ایجاد خواهد شد. به‌علاوه، سمافور دارای دسترسی خواندن و نوشتن برای سایر پردازه‌ها (از طریق پارامتر 0666) بوده و به ۱ مقداردهی می‌شود.

مزیت سمافورهای با نام این است که پردازه‌های متعدد غیرمرتبطی می‌توانند به‌راحتی از یک سمافور مشترک به‌عنوان یک راهکار همگامی به‌سادگی با ارجاع به نام سمافور استفاده کنند. در مثال بالا، به‌محض این که سمافور SEM ایجاد شد، فراخوانی‌های بعدی sem_open() (با پارامترهای یکسان) توسط سایر پردازه‌ها یک توصیف‌گری به سمافور موجود بر می‌گردانند.

ما در بخش ۶-۶، عملیات سمافور wait() و signal() کلاسیک را تعریف کردیم. پوزیکس این عملیات را به‌ترتیب با عملیات sem_wait() و sem_post() تعریف می‌کند. کد نمونه‌ی زیر، حفاظت از یک بخش بحرانی را با استفاده از سمافور با نام ایجاد شده در بالا نشان می‌دهد:

```
/* acquire the semaphore */
sem_wait(sem);

/* critical section */

/* release the semaphore */
sem_post(sem);
```

سیستم‌های لینوکس و macOS هر دو سمافورهای با نام پوزیکس را فراهم می‌کنند.

۷-۳-۲-۲ سمافورهای بی‌نام پوزیکس

کد زیر، تابع sem_init() را برای ایجاد و مقداردهی آغازین یک سمافور بی‌نام نشان می‌دهد که سه پارامتر به آن ارسال می‌شود:

۱- اشاره‌گری به سمافور
۲- پرچمی که سطح اشتراک را نشان می‌دهد.
۳- مقدار آغازین سمافور

و با مثال برنامه‌نویسی زیر نشان داده می‌شود:

```
#include <semaphore.h>
sem_t sem;

/* Create the semaphore and initialize it to 1 */
sem_init(&sem, 0, 1);
```

در این مثال، با ارسال پرچم ۰، نشان می‌دهیم که این سمافور می‌تواند فقط توسط نخ‌های متعلق به پردازه‌ای که سمافور را ایجاد کرده است به اشتراک گذاشته شود. (اگر یک مقدار غیرصفر ارسال می‌کردیم، می‌توانستیم به سمافور اجازه دهیم تا با گذاشتن آن در یک ناحیه‌ی حافظه‌ی مشترک بین پردازه‌های مجزایی به‌اشتراک گرفته شود.) به علاوه، سمافور را برابر ۱ مقداردهی می‌کنیم.

سمافورهای بی‌نام پوزیکس از عملیات sem_wait() و sem_post() به‌عنوان سمافورهای با نام استفاده می‌کنند. کد نمونه‌ی زیر، حفاظت یک بخش بحرانی را با استفاده از سمافوری که در بالا ایجاد شد نشان می‌دهد:

```
/* acquire the semaphore */
sem_wait(&sem);

/* critical section */

/* release the semaphore */
sem_post(&sem);
```

درست مثل قفل‌های انحصار متقابل، تمامی توابع سمافور، با موفقیت، مقدار ۰ و زمان وقوع یک شرط خطا، مقدار غیرصفر بر می‌گردانند.

۷-۳-۳ متغیرهای شرط پوزیکس

متغیرهای شرط در نخ‌های پوزیکس مشابه آن‌هایی که در بخش ۶-۷ بیان شدند رفتار می‌کنند. با این وجود، در آن بخش، متغیرهای شرط در میان چارچوب یک مانیتور استفاده می‌شوند که یک راهکار قفل‌گذاری برای تضمین جامعیت داده‌ها فراهم می‌کند. از آنجایی که نخ‌های پوزیکس

نوعاً در برنامه‌های C استفاده می‌شوند—و از آنجایی که C مانیتوری ندارد—ما قفل‌گذاری را با مرتبط‌ساختن یک متغیر شرط با یک قفل انحصار متقابل انجام می‌دهیم.
متغیرهای شرط در نخ‌های پوزیکس از نوع داده‌ی pthread_cond_t استفاده کرده و با استفاده از تابع pthread_cond_init() مقدار اولیه می‌گیرند. کد زیر یک متغیر شرط را ایجاد و مقدار اولیه می‌دهد، همین‌طور آن را با قفل انحصار متقابل مرتبط می‌سازد:

```
pthread_mutex_t mutex;
pthread_cond_t cond_var;

pthread_mutex_init(&mutex,NULL);
pthread_cond_init(&cond_var,NULL);
```

تابع pthread_cond_wait() برای انتظارکشی بر روی یک متغیر شرط استفاده می‌شود. کد زیر، چگونگی انتظارکشی یک نخ برای true شدن شرط a == b با استفاده از یک متغیر شرط نخ پوزیکس نشان می‌دهد:

```
pthread_mutex_lock(&mutex);

while (a != b)
       pthread_cond_wait(&cond_var, &mutex);

pthread_mutex_unlock(&mutex);
```

قفل انحصار متقابل مرتبط با متغیر شرط بایستی قبل از این که تابع pthread_cond_wait() فراخوانی شود، قفل شود، چون برای حفاظت داده‌های عبارت شرطی در مقابل یک شرایط مسابقه‌ی احتمالی استفاده می‌شود. همین که این قفل کسب شود، نخ می‌تواند شرط را بررسی کند. اگر شرط true نباشد، نخ در ادامه می‌تواند با ارسال قفل انحصار متقابل و متغیر شرط به‌عنوان پارامتر، تابع pthread_cond_wait() را احضار نماید. فراخوانی pthread_cond_wait() قفل انحصار متقابل را رها می‌سازد که با این کار، نخ دیگری اجازه پیدا می‌کند تا داده‌ی مشترک را دسترسی کرده و احتمالاً مقدار آن را به‌روز نماید تا این که عبارت شرط به true ارزیابی شود. (به‌منظور حفاظت در مقابل خطاهای برنامه، گذاشتن عبارت شرطی در میان یک حلقه مهم است تا این که شرط بعد از سیگنال‌دیدن دوباره بررسی شود.)

نخی که داده‌های مشترک را تغییر می‌دهد می‌تواند تابع pthread_cond_signal() را احضار کند که با این کار، به یک نخ منتظر متغیر شرط سیگنال می‌دهد. این موضوع در زیر نشان داده می‌شود:

```
pthread_mutex_lock(&mutex);
a = b;
pthread_cond_signal(&cond_var);
pthread_mutex_unlock(&mutex);
```